

- Dispensa — Progetto 2 Principiante
 - Quiz a Risposta Multipla
 - Indice
 - 1. Liste: creazione, accesso, len(), iterazione
 - Creazione
 - Accesso per indice
 - len() — lunghezza
 - .append() — aggiungere alla fine
 - Iterazione con for
 - Contare con un ciclo
 - 2. Dizionari: creazione e accesso per chiave
 - Nota: il valore di una chiave può essere una lista!
 - .items() — iterare su chiavi e valori
 - 3. Liste di dizionari
 - Nel progetto: le domande del quiz
 - Accesso
 - Iterazione
 - Pensala come una tabella
 - 4. Cicli: for con enumerate() e range()
 - range() — generare sequenze di numeri
 - enumerate() — indice + elemento
 - enumerate() nel progetto
 - enumerate() + zip() combinati
 - 5. zip() — iterare su due liste in parallelo
 - Come funziona
 - Nel progetto: domande + risposte
 - 6. Condizionali: if / elif / else
 - Ripasso
 - Nel progetto: valutazione del punteggio
 - Validazione con in e and
 - 7. Funzioni: def, parametri, return, valori di default
 - Parametri con valori di default
 - Restituire un dizionario
 - 8. F-string e formattazione avanzata
 - Ripasso base
 - Formattazione decimali
 - Centrare un titolo

- Ripetizione di stringhe
- Calcolare la barra visuale
- 9. `ord()` e `chr()` — convertire tra lettere e numeri
 - `ord()` — da carattere a numero
 - `chr()` — da numero a carattere
 - Conversione lettera ↔ indice
 - Nel progetto
- 10. Operatori di confronto
 - Confronto tra stringhe
 - `not in` — negazione di `in`
- 11. Calcoli con percentuali
 - Formula base
 - Protezione dalla divisione per zero
 - Conversione in intero (per la barra visuale)
- 12. Riepilogo e consigli
 - Mappa concetti → dove li usi nel progetto
 - Ordine consigliato di implementazione
 - Errori comuni da evitare
 - Riferimenti ufficiali

Dispensa — Progetto 2 Principiante

Quiz a Risposta Multipla

Obiettivo della dispensa: fornirti gli strumenti necessari per completare il progetto. In alcuni casi faremo riferimenti espliciti al codice che verrà usato direttamente nella soluzione.

Indice

1. Liste: creazione, accesso, `len()`, iterazione
2. Dizionari: creazione e accesso per chiave
3. Liste di dizionari
4. Cicli: `for` con `enumerate()` e `range()`

5. `zip()` — iterare su due liste in parallelo
 6. Condizionali: `if` / `elif` / `else`
 7. Funzioni: `def`, parametri, `return`, valori di default
 8. F-string e formattazione avanzata
 9. `ord()` e `chr()` — convertire tra lettere e numeri
 10. Operatori di confronto
 11. Calcoli con percentuali
 12. Riepilogo e consigli
-

1. Liste: creazione, accesso, `len()`, iterazione

Una **lista** è una sequenza ordinata di elementi. Gli elementi possono essere di qualsiasi tipo e sono indicizzati a partire da **0**.

Creazione

```
# Lista vuota
numeri = []

# Lista con elementi
frutti = ["mela", "banana", "arancia"]
voti = [28, 30, 25, 27]
mista = ["ciao", 42, True, 3.14] # tipi diversi nella stessa lista
```

Accesso per indice

```
frutti = ["mela", "banana", "arancia", "kiwi"]
#          [0]      [1]      [2]      [3]
#          [-4]     [-3]     [-2]     [-1]

frutti[0]    # "mela"      (primo elemento)
frutti[2]    # "arancia"   (terzo elemento)
frutti[-1]   # "kiwi"     (ultimo elemento)
frutti[-2]   # "arancia"  (penultimo)
```

Regola: gli indici partono da 0. L'elemento con indice **N** è il **(N+1)-esimo** elemento.

len() — lunghezza

```
frutti = ["mela", "banana", "arancia"]
print(len(frutti))    # 3

# L'ultimo indice valido è len(lista) - 1
# frutti[3] → IndexError! (solo indici 0, 1, 2)
```

.append() — aggiungere alla fine

```
risultati = []
risultati.append(True)    # [True]
risultati.append(False)  # [True, False]
risultati.append(True)    # [True, False, True]
```

Iterazione con for

```
frutti = ["mela", "banana", "arancia"]

for frutto in frutti:
    print(frutto)
# mela
# banana
# arancia
```

Contare con un ciclo

```
numeri = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

contatore = 0
for n in numeri:
    if n % 2 == 0:    # se n è pari
        contatore += 1    # equivale a: contatore = contatore + 1

print(f"Numeri pari: {contatore}")    # 5
```

2. Dizionari: creazione e accesso per chiave

Un **dizionario** (**dict**) associa **chiavi** a **valori**.

```
domanda = {
    "testo": "Quale funzione stampa a schermo?",
    "opzioni": ["echo()", "write()", "console.log()", "print()"],
    "corretta": 3
}

# Accesso per chiave
print(domanda["testo"])      # "Quale funzione stampa a schermo?"
print(domanda["corretta"])   # 3
print(domanda["opzioni"])    # ["echo()", "write()", "console.log()", "print()"]
print(domanda["opzioni"][3]) # "print()" (accesso alla lista dentro il dict)
```

Nota: il valore di una chiave può essere una lista!

Nel progetto, **"opzioni"** è una chiave il cui valore è una **lista di stringhe**. Per accedere a una singola opzione, usiamo due livelli di indicizzazione:

```
# Livello 1: dizionario["chiave"] → ottieni la lista
# Livello 2: lista[indice] → ottieni l'elemento
domanda["opzioni"][0]    # "echo()"
domanda["opzioni"][3]    # "print()"
```

.items() — iterare su chiavi e valori

```
risultati = {"corrette": 4, "totale": 5, "percentuale": 80.0}

for chiave, valore in risultati.items():
    print(f"{chiave}: {valore}")
```

3. Liste di dizionari

Una **lista di dizionari** è una delle strutture dati più comuni in Python. Ogni elemento della lista è un dizionario con le stesse chiavi.

Nel progetto: le domande del quiz

```
domande = [  
    {  
        "testo": "Quale funzione stampa a schermo?",  
        "opzioni": ["echo()", "write()", "console.log()", "print()"],  
        "corretta": 3  
    },  
    {  
        "testo": "Quale tipo di dato rappresenta 3.14?",  
        "opzioni": ["str", "int", "bool", "float"],  
        "corretta": 3  
    },  
    {  
        "testo": "Come si crea una lista vuota?",  
        "opzioni": ["list = {}", "list = ()", "list = ''", "lista = []"],  
        "corretta": 3  
    }  
]
```

Accesso

```
# Prima domanda (indice 0 nella lista)  
domande[0]                # il dizionario della prima domanda  
domande[0]["testo"]        # "Quale funzione stampa a schermo?"  
domande[0]["opzioni"][3]   # "print()"  
  
# Seconda domanda (indice 1)  
domande[1]["testo"]        # "Quale tipo di dato rappresenta 3.14?"  
domande[1]["corretta"]     # 3
```

Iterazione

```
for domanda in domande:  
    print(domanda["testo"])
```

```
print(f" Risposta corretta: {domanda['opzioni'][domanda['corretta']]}")
```

Pensala come una tabella

#	testo	opzioni	corretta
0	"Quale funzione stampa..."	["echo()", "write()", ...]	3
1	"Quale tipo di dato..."	["str", "int", ...]	3
2	"Come si crea una lista..."	["list = {}", ...]	3

Ogni **riga** è un dizionario, l'intera **tabella** è una lista.

Riferimento: [Data Structures](#)

4. Cicli: **for** con **enumerate()** e **range()**

range() — generare sequenze di numeri

```
# range(stop) → da 0 a stop-1
for i in range(5):
    print(i)          # 0, 1, 2, 3, 4

# range(start, stop) → da start a stop-1
for i in range(1, 6):
    print(i)          # 1, 2, 3, 4, 5

# range(start, stop, step) → con incremento
for i in range(0, 10, 2):
    print(i)          # 0, 2, 4, 6, 8
```

enumerate() — indice + elemento

Quando iteriamo su una lista e ci serve anche l'**indice**, usiamo **enumerate()**:

```
frutti = ["mela", "banana", "arancia"]
```

```
# SENZA enumerate: solo l'elemento
for frutto in frutti:
    print(frutto)                # mela, banana, arancia

# CON enumerate: indice + elemento
for indice, frutto in enumerate(frutti):
    print(f"{indice}: {frutto}")

# 0: mela
# 1: banana
# 2: arancia

# Con partenza da 1 (utile per la numerazione visuale)
for numero, frutto in enumerate(frutti, 1):
    print(f"{numero}. {frutto}")

# 1. mela
# 2. banana
# 3. arancia
```

enumerate() nel progetto

```
# Mostrare le opzioni con le lettere a, b, c, d
for i, opzione in enumerate(domanda["opzioni"]):
    lettera = chr(i + ord("a"))    # 0→'a', 1→'b', 2→'c', 3→'d'
    print(f" {lettera} {opzione}")
```

Output:

```
a) echo()
b) write()
c) console.log()
d) print()
```

enumerate() + zip() combinati

```
for i, (domanda, esito) in enumerate(zip(domande, dettaglio)):
    simbolo = "+" if esito else "x"
    print(f" {i+1}. [{simbolo}] {domanda['testo']}")
```

Questo è un pattern avanzato: **enumerate** fornisce l'indice **i**, **zip** accoppia domanda ed esito. Le parentesi intorno a **(domanda, esito)** sono necessarie per l'unpacking della tupla restituita da **zip**.

5. `zip()` — iterare su due liste in parallelo

`zip()` accoppia gli elementi di due (o più) liste **per posizione**:

```
nomi = ["Mario", "Anna", "Luca"]
voti = [28, 30, 25]

for nome, voto in zip(nomi, voti):
    print(f"{nome}: {voto}")
# Mario: 28
# Anna: 30
# Luca: 25
```

Come funziona

```
zip(["a", "b", "c"], [1, 2, 3])
# produce le coppie: ("a", 1), ("b", 2), ("c", 3)
```

Se le liste hanno lunghezze diverse, `zip` si ferma alla più corta.

Nel progetto: domande + risposte

```
domande = [...]          # lista di 5 domande
risposte = ["d", "d", "d", "d", "b"] # lista di 5 risposte

for domanda, risposta in zip(domande, risposte):
    esito = verifica_risposta(domanda, risposta)
    # ...
```

`zip` accoppia la prima domanda con la prima risposta, la seconda con la seconda, ecc.

6. Condizionali: **if** / **elif** / **else**

Ripasso

```
if condizione_1:
    # ...
elif condizione_2:
    # ...
else:
    # ...
```

Nel progetto: valutazione del punteggio

```
def valutazione(percentuale):
    if percentuale >= 90:
        return "Eccellente!"
    elif percentuale >= 70:
        return "Ottimo!"
    elif percentuale >= 50:
        return "Sufficiente"
    else:
        return "Da ripassare..."
```

L'ordine conta! Le condizioni sono verificate dall'alto verso il basso. Se **percentuale** è 95, la prima condizione (**>= 90**) è già **True** e le altre vengono saltate.

Validazione con **in** e **and**

```
risposta = "b"

# Verificare che sia una lettera valida
if risposta in "abcd" and len(risposta) == 1:
    print("Risposta valida")
else:
    print("Risposta non valida")
```

- **risposta in "abcd"** → **True** se la stringa è contenuta in "abcd"
- **and** → entrambe le condizioni devono essere **True**

- `len(risposta) == 1` → la stringa deve avere esattamente un carattere
-

7. Funzioni: `def`, parametri, `return`, valori di default

Parametri con valori di default

```
def calcola_risultati(domande, risposte, punti_per_domanda=10):  
    #  
    #  
    punteggio = corrette * punti_per_domanda  
    # ...
```

Chiamate possibili:

```
# Usa il default (10 punti per domanda)  
calcola_risultati(domande, risposte)  
  
# Sovrascrive il default  
calcola_risultati(domande, risposte, 20)  
  
# Con keyword argument (più esplicito)  
calcola_risultati(domande, risposte, punti_per_domanda=20)
```

Restituire un dizionario

Una funzione può restituire un **dizionario** con più valori:

```
def calcola_risultati(domande, risposte, punti_per_domanda=10):  
    corrette = 0  
    dettaglio = []  
  
    for domanda, risposta in zip(domande, risposte):  
        esito = verifica_risposta(domanda, risposta)  
        dettaglio.append(esito)  
        if esito:  
            corrette += 1  
  
    totale = len(domande)
```

```
return {
    "corrette": corrette,
    "totale": totale,
    "punteggio": corrette * punti_per_domanda,
    "punteggio_max": totale * punti_per_domanda,
    "percentuale": (corrette / totale * 100) if totale > 0 else 0,
    "dettaglio": dettaglio
}
```

Uso:

```
risultati = calcola_risultati(domande, risposte)
print(risultati["corrette"])      # 4
print(risultati["percentuale"])  # 80.0
print(risultati["dettaglio"])     # [True, True, True, True, False]
```

8. F-string e formattazione avanzata

Ripasso base

```
nome = "Mario"
punti = 40
print(f"Bravo {nome}! Hai {punti} punti!")
```

Formattazione decimali

```
percentuale = 80.0
print(f"Percentuale: {percentuale:.1f}%")  # "80.0%"
print(f"Percentuale: {percentuale:.0f}%")  # "80%"
```

Centrare un titolo

```
print(f"{'RISULTATI FINALI':^50}")
```

La sintassi `{valore:^N}` centra `valore` in un campo di `N` caratteri.

Ripetizione di stringhe

```
"=" * 50    # "===== "  
"# " * 10   # "##### "  
  
# Creare una barra di progresso  
barra_piena = "# " * 16    # "##### "  
barra_vuota = "- " * 4     # "---- "  
print(f"[{barra_piena}{barra_vuota}] 80%")  
# [#####----] 80%
```

Calcolare la barra visuale

```
percentuale = 80.0  
barra_piena = int(percentuale / 5)    # 80 / 5 = 16 blocchi  
barra_vuota = 20 - barra_piena        # 20 - 16 = 4 blocchi  
print(f"[{'#' * barra_piena}{ '-' * barra_vuota}] {percentuale:.0f}%")  
# [#####----] 80%
```

Riferimento: [Format Specification](#)

9. `ord()` e `chr()` — convertire tra lettere e numeri

Ogni carattere ha un **codice numerico** (codice ASCII/Unicode). Le funzioni `ord()` e `chr()` convertono tra carattere e codice.

`ord()` — da carattere a numero

```
ord("a")    # 97  
ord("b")    # 98
```

```
ord("c")    # 99
ord("d")    # 100
ord("A")    # 65
ord("0")    # 48
```

chr() — da numero a carattere

```
chr(97)     # "a"
chr(98)     # "b"
chr(99)     # "c"
chr(100)    # "d"
chr(65)     # "A"
```

Conversione lettera ↔ indice

Nel progetto, le opzioni sono indicizzate 0-3 ma l'utente risponde con una lettera a-d.

Dobbiamo convertire tra i due formati:

```
# Lettera → Indice
# ord("a") - ord("a") = 97 - 97 = 0
# ord("b") - ord("a") = 98 - 97 = 1
# ord("c") - ord("a") = 99 - 97 = 2
# ord("d") - ord("a") = 100 - 97 = 3

risposta = "b"
indice = ord(risposta) - ord("a")    # 1

# Indice → Lettera
# chr(0 + ord("a")) = chr(97) = "a"
# chr(1 + ord("a")) = chr(98) = "b"
# chr(2 + ord("a")) = chr(99) = "c"
# chr(3 + ord("a")) = chr(100) = "d"

indice = 3
lettera = chr(indice + ord("a"))     # "d"
```

Nel progetto

```
# Mostrare le opzioni con lettere
for i, opzione in enumerate(domanda["opzioni"]):
    lettera = chr(i + ord("a"))
```

```
print(f" {lettera}} {opzione}")
```

```
# Verificare la risposta
```

```
def verifica_risposta(domanda, risposta_utente):  
    indice = ord(risposta_utente.lower()) - ord("a")  
    return indice == domanda["corretta"]
```

Riferimento: [ord\(\)](#) — [chr\(\)](#)

10. Operatori di confronto

Operatore	Significato	Esempio
<code>==</code>	Uguale a	<code>indice == 3 → True</code>
<code>!=</code>	Diverso da	<code>risposta != "a" → True</code>
<code>>=</code>	Maggiore o uguale	<code>percentuale >= 90 → True</code> se ≥ 90
<code><</code>	Minore di	<code>percentuale < 50 → True</code> se < 50

Confronto tra stringhe

Le stringhe possono essere confrontate con `==`:

```
"a" == "a"      # True  
"a" == "A"      # False! (case-sensitive)  
"a" == "b"      # False
```

Per questo nel progetto usiamo `.lower()` prima del confronto.

`not in` — negazione di `in`

```
risposta = "x"  
if risposta not in "abcd":  
    print("Risposta non valida!")
```

11. Calcoli con percentuali

Formula base

```
percentuale = (parte / totale) * 100
```

```
corrette = 4
totale = 5
percentuale = (corrette / totale) * 100    # 80.0
```

Protezione dalla divisione per zero

Se **totale** è 0, la divisione causa **ZeroDivisionError**. Usiamo l'operatore ternario:

```
percentuale = (corrette / totale * 100) if totale > 0 else 0
```

Questa riga significa:

- **Se** **totale** > 0 → calcola **(corrette / totale * 100)**
- **Altrimenti** → il risultato è 0

Conversione in intero (per la barra visuale)

```
percentuale = 80.0
blocchi = int(percentuale / 5)    # int(16.0) = 16

# int() TRONCA il decimale (non arrotonda!)
int(16.9)    # 16 (non 17!)
int(16.1)    # 16
```

12. Riepilogo e consigli

Mappa concetti → dove li usi nel progetto

Concetto	Dove lo usi
Lista di dizionari	Le domande del quiz
Accesso <code>dict["chiave"]</code>	Ogni volta che leggi testo, opzioni, risposta corretta
<code>enumerate()</code>	Mostrare le opzioni con lettere (a, b, c, d)
<code>zip()</code>	Accoppiare domande e risposte per la correzione
<code>ord()</code> / <code>chr()</code>	Convertire tra lettere (a-d) e indici (0-3)
<code>if/elif/else</code>	Valutazione del punteggio
Default <code>punti_per_domanda=10</code>	Parametro opzionale in <code>calcola_risultati()</code>
Return dizionario	<code>calcola_risultati()</code> restituisce i dati aggregati
<code>:.1f</code>	Percentuale con 1 decimale
<code>{:^50}</code>	Titolo centrato
<code>"#" * N</code>	Barra visuale del progresso
<code>.lower()</code> / <code>.strip()</code>	Pulire la risposta dell'utente

Ordine consigliato di implementazione

1. `crea_domande()` — definisci la struttura dati (lista di dizionari)
2. `mostra_domanda()` — stampa una domanda con le opzioni (usa `enumerate` e `chr`)
3. `verifica_risposta()` — converti lettera→indice con `ord()` e confronta
4. `valutazione()` — `if/elif/else` sulla percentuale
5. `calcola_risultati()` — usa `zip()` per iterare domande+risposte
6. `mostra_risultati()` — stampa formattata con f-string
7. **Metti tutto insieme** e testa con risposte simulate

Errori comuni da evitare

- **Indice vs lettera:** le opzioni sono indicizzate 0-3, ma l'utente vede a-d. Ricorda la conversione.
 - **enumerate parte da 0:** usa `enumerate(lista, 1)` se vuoi mostrare "Domanda 1/5" invece di "Domanda 0/5".
 - **Divisione per zero:** proteggi `corrette / totale` con un controllo.
 - **Dimenticare `.lower()`:** "A" ≠ "a", quindi normalizza la risposta dell'utente.
-

Riferimenti ufficiali

- [Liste e strutture dati](#)
- `enumerate()`
- `zip()`
- `ord()` e `chr()`
- [Formattazione stringhe](#)
- [Operatore ternario](#)